

Python Object-Oriented Programming (OOP)

Complete Study Notes — Beginner to Intermediate

Classes, objects, attributes, methods, self, access control, properties, class/static members, encapsulation, abstraction, inheritance, polymorphism, ABC, and practical revision.

1. Classes and Objects

A class is a blueprint for creating objects. An object is an instance of a class.

```
class Employee:
    pass

emp1 = Employee()
emp2 = Employee()
```

2. Attributes

Attributes are data associated with an object or class. Instance attributes can differ between objects.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

emp = Employee("VK", 30000)
print(emp.name)
```

3. Methods and self

A method is a function defined inside a class. self refers to the current instance.

```
class Employee:
    def __init__(self, name):
        self.name = name

    def display(self):
        print(self.name)

emp = Employee("VK")
emp.display()
```

4. __init__

`__init__` is an initializer automatically called when an object is initialized. It commonly sets instance attributes.

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

s = Student("VK", 21)
```

5. Public, Protected and Private

Python uses naming conventions rather than strict access modifiers. Public: name. Protected convention: `_name`. Private/name-mangled: `__name`.

```
class Account:
    public_value = 10
    _protected_value = 20
    __private_value = 30
```

6. Getters, Setters and @property

Properties provide controlled access to internal state and allow validation.

```
class Employee:
    def __init__(self, salary):
        self._salary = salary
```

```

@property
def salary(self):
    return self._salary

@salary.setter
def salary(self, value):
    if value >= 0:
        self._salary = value
    else:
        raise ValueError("Salary cannot be negative")

```

7. Class Attributes

A class attribute is shared through the class unless an instance overrides it.

```

class Employee:
    company = "ABC Ltd"

    def __init__(self, name):
        self.name = name

```

8. Class Methods

A class method uses `@classmethod` and receives `cls`, referring to the class.

```

class Employee:
    company = "ABC Ltd"

    @classmethod
    def change_company(cls, name):
        cls.company = name

```

9. Static Methods

A static method uses `@staticmethod` and does not automatically receive `self` or `cls`. It is useful for helper logic related to the class.

```

class MathTools:
    @staticmethod
    def add(a, b):
        return a + b

print(MathTools.add(5, 3))

```

10. Encapsulation

Encapsulation bundles data and behavior and controls how internal state is changed.

```

class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def get_balance(self):
        return self.__balance

```

11. Abstraction and ABC

Abstraction exposes essential behavior while hiding implementation details. ABC means Abstract Base Class.

`@abstractmethod` defines operations subclasses must implement.

```

from abc import ABC, abstractmethod

class Vehicle(ABC):
    @abstractmethod
    def start(self):
        pass

class Car(Vehicle):
    def start(self):

```

```
        print("Car started")

car = Car()
car.start()
```

12. Inheritance

Inheritance lets a child class reuse or extend a parent class. Common forms include single, multilevel, hierarchical and multiple inheritance.

```
class Employee:
    def work(self):
        print("Working")

class Developer(Employee):
    def code(self):
        print("Coding")

dev = Developer()
dev.work()
dev.code()
```

13. super()

super() is commonly used to call parent-class methods, especially a parent initializer.

```
class Employee:
    def __init__(self, name):
        self.name = name

class Developer(Employee):
    def __init__(self, name, language):
        super().__init__(name)
        self.language = language
```

14. Method Overriding

A child class can replace an inherited method with its own implementation.

```
class Animal:
    def sound(self):
        print("Some sound")

class Dog(Animal):
    def sound(self):
        print("Bark")
```

15. Polymorphism

Polymorphism allows different objects to respond to the same operation in their own way. Python also supports duck typing.

```
class Dog:
    def speak(self):
        print("Bark")

class Cat:
    def speak(self):
        print("Meow")

for animal in (Dog(), Cat()):
    animal.speak()
```

16. Four Pillars of OOP

Encapsulation: bundle data/behavior and control state. Abstraction: hide unnecessary implementation details.

Inheritance: reuse/extend a parent class. Polymorphism: common operations can have different implementations.

17. Composition vs Inheritance

Composition is a 'has-a' relationship; inheritance is generally an 'is-a' relationship. Prefer composition when an object should use another object rather than specialize it.

```
class Engine:
    def start(self):
        print("Engine started")

class Car:
    def __init__(self):
        self.engine = Engine()

    def start(self):
        self.engine.start()
```

18. Magic Methods

Dunder methods integrate objects with Python operations. Common examples include `__str__`, `__repr__`, `__eq__`, `__len__`, and `__add__`.

```
class Student:
    def __init__(self, name):
        self.name = name

    def __str__(self):
        return self.name

    def __eq__(self, other):
        return self.name == other.name
```

19. OOP Best Practices

Keep classes focused. Use meaningful names. Avoid unnecessary inheritance. Prefer composition where appropriate. Validate important state changes. Practice each concept with small programs.

20. Interview Quick Revision

Class = blueprint. Object = instance. `self` = current instance. `__init__` = initializer. Encapsulation = bundle/control state. Abstraction = hide implementation details. Inheritance = derive/reuse. Polymorphism = different behavior through a common operation. ABC = Abstract Base Class. Class method receives `cls`; static method receives neither `self` nor `cls` automatically.

21. Practice Roadmap

1) Student/Employee classes. 2) Constructors and methods. 3) Properties and validation. 4) Inheritance. 5) Polymorphism. 6) ABC. 7) Magic methods. 8) Composition. 9) Rebuild a real project using OOP.

Key idea: Don't memorize OOP definitions. Write classes, modify them, break them, debug them, and apply each concept to a real project.